

UNIVERSIDAD DE CONCEPCIÓN
FACULTAD DE INGENIERÍA
DEPARTAMENTO DE INGENIERÍA INFORMÁTICA Y CIENCIAS DE LA COMPUTACIÓN

TAREA 1 - FORMA M
Sistemas de arriendo de equipos

INTEGRACIÓN DE SISTEMAS (501451-1)

Grupo 7

POR
Carlos T. Alvarez Norambuena
Antonio J. Benavides Puentes
Martin A. Tapia Medina
Pablo E. Villagran Hermanns

Profesor: Gonzalo E. Pérez Correa

Septiembre 2026
Concepción (Chile)

Índice

1 Análisis del problema **1**

1.1 Contexto 1

1.2 Problema 1

1.3 Solución propuesta 2

1.4 Restricciones y criterios de diseño 3

2 Diseño de la solución **3**

3 Architecture Decision Records (ADRs) **6**

4 Experimento **12**

4.1 Objetivo e hipótesis 12

4.2 Diseño y métricas 12

4.3 Resultados 13

4.4 Prueba estadística 14

4.5 Interpretación y limitaciones 14

4.6 Conclusiones 15

5 Consideraciones de contexto **15**

6 Instrucciones de ejecución **16**

6.1 Ejecución de los servicios 16

6.2 Ejecución de las pruebas 17

6.3 Verificación de contenedores activos 17

7 Reflexión final **18**

Referencias **19**

1 Análisis del problema

1.1 Contexto

La empresa de arriendo de maquinaria opera actualmente con dos sistemas independientes: uno encargado de administrar el catálogo de equipos y su disponibilidad, y otro responsable de gestionar clientes y contratos de arriendo. Debido a la ausencia de integración entre ambos sistemas, el servicio de Arriendos no puede verificar de forma confiable la disponibilidad de una unidad antes de registrar una operación.

Esta separación genera el riesgo de que la información registrada en los arriendos no corresponda con la disponibilidad real de los equipos. Por ejemplo, una unidad podría asignarse a más de un arriendo o permanecer registrada como ocupada después de que una operación haya sido cancelada. Estas situaciones pueden afectar la planificación de las entregas y la consistencia de la información administrada por ambos sistemas.

Por lo tanto, el problema no se limita a establecer un mecanismo de comunicación entre los servicios, sino que también requiere considerar la coordinación de las operaciones que modifican la disponibilidad de los equipos y el estado de los arriendos.

1.2 Problema

Se requiere integrar ambos sistemas mediante APIs con responsabilidades y protocolos diferenciados. El sistema de Arriendos debe exponer una API REST para sus consumidores externos, mientras que el sistema de Equipos debe proporcionar una API gRPC para la comunicación interna.

La integración debe permitir que una solicitud de arriendo valide y reserve una unidad disponible en el sistema de Equipos antes de registrar la operación en el sistema de Arriendos. Asimismo, las operaciones de liberación deben actualizar la disponibilidad de las unidades cuando un arriendo se cancela o cuando corresponde finalizar la reserva.

Sin embargo, esta integración presenta diversas restricciones. En primer lugar, las solicitudes pueden ejecutarse de manera concurrente. Dos solicitudes de arriendo podrían consultar la disponibilidad de una misma unidad antes de que alguna complete la reserva. Si la verificación y la actualización de la disponibilidad no se realizan de manera coordinada, ambas solicitudes podrían intentar reservar la misma unidad, provocando una sobreasignación.

En segundo lugar, las operaciones de reserva o liberación y el registro del arriendo afectan a sistemas con bases de datos independientes. Por lo tanto, no existe necesariamente una transacción atómica que abarque ambas operaciones. Si la reserva se completa en Equipos, pero el registro del arriendo falla en Arriendos, la unidad podría permanecer reservada sin un arriendo asociado. De manera inversa, si una liberación se ejecuta correctamente, pero falla la actualización correspondiente en Arriendos, los sistemas podrían representar estados

diferentes para una misma operación.

Un tercer aspecto corresponde a los fallos de comunicación y los tiempos de espera agotados. Una solicitud gRPC podría ejecutarse correctamente en Equipos, pero su respuesta no llegar a Arriendos. En ese escenario, el servicio solicitante no puede determinar únicamente a partir del error de comunicación si la operación remota se ejecutó o no. Repetir la solicitud sin considerar esta posibilidad podría generar efectos duplicados o resultados inesperados.

En consecuencia, el diseño debe considerar no solo la disponibilidad de los servicios, sino también la concurrencia, la consistencia entre las bases de datos, la recuperación ante fallos parciales y la seguridad de los reintentos. El uso de REST y gRPC permite establecer contratos de comunicación diferenciados, pero no garantiza por sí mismo la consistencia de las operaciones distribuidas.

1.3 Solución propuesta

Se decidió implementar ambos servicios de forma independiente, cada uno con su propia base de datos y contrato de comunicación. El servicio de Arriendos se implementó mediante FastAPI y expone una API REST versionada para sus consumidores externos, mientras que el servicio de Equipos utiliza gRPC para la comunicación interna y la gestión de las operaciones relacionadas con la disponibilidad.

Los contratos de integración se definen explícitamente mediante OpenAPI para la API REST y Protocol Buffers para gRPC.

El flujo de creación de un arriendo contempla una solicitud al servicio de Equipos para reservar una unidad antes de registrar el arriendo en la base de datos de Arriendos. Cuando el registro no puede completarse después de una reserva exitosa, se requiere gestionar la liberación de la unidad para evitar que permanezca reservada sin una operación asociada. De manera análoga, la cancelación o finalización de un arriendo debe coordinar la actualización del registro correspondiente con la liberación de la unidad.

Como mecanismos complementarios, se incorporó autenticación mediante API Key, con claves diferenciadas según los permisos de lectura y escritura. Además, se utiliza una caché con Redis para reducir consultas repetitivas de arriendos y se implementaron pruebas automatizadas orientadas a verificar las operaciones principales del sistema.

La caché contribuye a reducir la carga de consultas, pero requiere considerar la invalidación o actualización de las entradas cuando cambia el estado de un arriendo. De lo contrario, una respuesta almacenada podría representar información desactualizada.

Adicionalmente, se implementó un segundo cliente gRPC en Node.js que utiliza el mismo contrato `equipos.proto`, con el propósito de demostrar la interoperabilidad del servicio y la independencia entre la implementación del cliente y la tecnología utilizada para desarrollar el servidor.

Finalmente, ambos servicios se ejecutan de forma independiente mediante contenedores Docker. También se contempla el manejo de fallas en la comunicación entre Arriendos y Equipos, con el objetivo de evitar que una indisponibilidad temporal del servicio interno provoque respuestas incontroladas en la API pública.

1.4 Restricciones y criterios de diseño

A partir del análisis anterior, se identifican los siguientes criterios para evaluar la solución:

- **Disponibilidad de las unidades:** una reserva debe validar y actualizar la disponibilidad de manera coordinada, evitando que solicitudes concurrentes asignen una misma unidad más veces de las permitidas.
- **Consistencia entre servicios:** el registro de un arriendo y la reserva de la unidad deben coordinarse considerando que pertenecen a bases de datos independientes y que una de las operaciones puede fallar después de que la otra haya finalizado.
- **Recuperación ante fallos parciales:** cuando una operación se completa en un servicio y falla en el otro, debe existir una estrategia definida para recuperar la consistencia.
- **Reintentos e idempotencia:** los reintentos ante fallos de comunicación o tiempos de espera agotados deben considerar la posibilidad de que la operación original haya sido ejecutada. Para evitar efectos duplicados o estados inconsistentes, las operaciones deben permitir identificar o gestionar solicitudes repetidas de forma segura.
- **Consistencia de la caché:** las operaciones que modifican los arriendos deben considerar la invalidación o actualización de las entradas correspondientes para evitar que las consultas devuelvan información desactualizada.
- **Aislamiento de responsabilidades:** Arriendos debe gestionar los clientes y contratos, mientras que Equipos debe mantener la información relativa a las unidades y su disponibilidad. La comunicación entre ambos debe respetar los contratos definidos para REST y gRPC.
- **Observabilidad y pruebas:** las pruebas deben considerar tanto los flujos exitosos como los errores de comunicación, las solicitudes concurrentes y los fallos producidos entre la reserva y el registro del arriendo.

2 Diseño de la solución

La solución se divide en los servicios **Arriendos** y **Equipos** usando como frontera sus responsabilidades de negocio y la propiedad de los datos, en lugar de separar solo por capas técnicas. Se descartó un monolito porque los sistemas ya representan contextos distintos

y tienen responsables propios: Arriendos gestiona clientes y contratos, mientras Equipos mantiene la fuente oficial del catálogo y de las unidades disponibles. Por eso, Arriendos conserva el identificador del equipo asociado a cada arriendo, pero no accede a la base de Equipos; solicita las reservas y liberaciones mediante gRPC. Esta separación evita acoplar las bases de datos y permite evolucionar cada servicio por su cuenta, a cambio de depender de la red y tener que manejar fallas o lentitud en la comunicación. La duplicación deliberada se limita a los resultados que Redis conserva temporalmente para acelerar consultas; como la caché puede quedar desactualizada, se invalida después de las mutaciones y no se considera la fuente oficial de los datos.

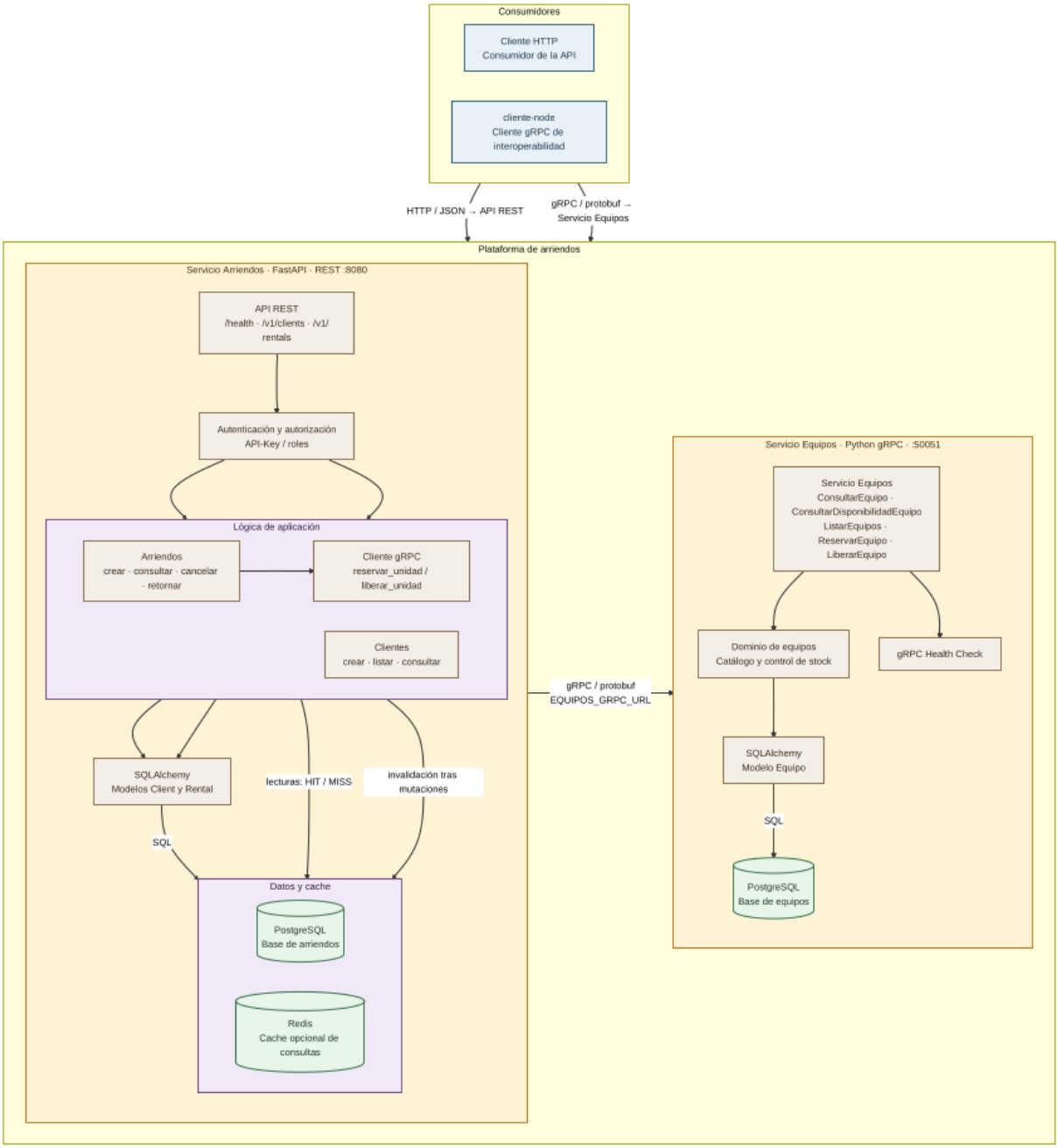


Figura 1: Arquitectura aplicacion

Para controlar las cancelaciones y los retornos simultáneos de un mismo arriendo, se incorporó una actualización condicional que permite cambiar su estado únicamente si permanece en ACTIVE. Solo la solicitud que realiza esa transición continúa con la liberación de la unidad; una vez confirmada, las demás reciben HTTP 409, evitando liberar varias unidades por un solo arriendo. Asimismo, se incorporó la traducción del código gRPC $FAILED_PRECONDITION$ a HTTP 409. Esto permite informar un conflicto cuando otra solicitud reserva la última unidad entre la consulta de disponibilidad y el intento de reserva, en lugar de responder con un error 500.

3 Architecture Decision Records (ADRs)

ADR-001 - Estilo de integración y descomposición. Implementación de microservicios por sobre un monolito.

Estado: Aceptada

Contexto. El sistema gestiona equipos de arriendo y el ciclo de vida de los arriendos. Ambas responsabilidades podrían vivir en un único servicio monolítico con una sola base de datos, en dos servicios que compartan base de datos, o en dos servicios independientes con datos propios que se comunican por gRPC, como está implementado actualmente. La decisión es necesaria porque el servicio de Equipos puede ser reutilizado por otros consumidores además de Arriendos, y porque el flujo de arriendos no debería depender del ciclo de cambios del inventario.

Alternativas consideradas.

- *Opción A* – Integrar todo como un gran monolito, con una única base de datos. Permite transacciones ACID entre reserva y arriendo, evita replicación de datos innecesaria y plantea una arquitectura más simple. Cuesta acoplar los ciclos de vida, el escalamiento y los fallos de ambos dominios.
- *Opción B* – Monolito modular (un solo despliegue, módulos con límites internos). Separa la lógica sin perder transacciones. Cuesta que los límites se sostienen solo por disciplina del equipo y que sigue habiendo un único proceso que escalar y que puede caer.
- *Opción C* – Dos servicios con base de datos compartida. Permite desplegar por separado, mantener la transacción local y una llave foránea real sobre `equipment_id`. Cuesta que el esquema se convierte en un contrato implícito y que ambos servicios pueden escribir las mismas tablas.
- *Opción D* – Integrar todo separado en dos microservicios, con una base de datos propia de cada uno. Mejora la separación de la lógica de negocio, da propiedad exclusiva de los datos y permite un mejor desarrollo en paralelo. Cuesta perder la transacción entre ambos servicios y la llave foránea real sobre `equipment_id`.

Decisión. Se eligió separar en dos microservicios, uno para el manejo de los arriendos y otro para el manejo de los equipos, cada uno dueño exclusivo de su base de datos y comunicados por gRPC (`reservar_unidad` y `liberar_unidad`) para el arriendo y la liberación de equipos.

Justificación. La frontera entre servicios sigue la responsabilidad y la propiedad de los datos: Equipos es dueño de la existencia, los atributos y la disponibilidad de cada unidad, y Arriendos es dueño de los clientes, los arriendos y su ciclo de vida (`ACTIVE`, `COMPLETED`, `CANCELLED`). Cada dato tiene un único servicio que puede escribirlo, y el otro solo lo referencia o lo consulta por API. El criterio es quién escribe el dato, no quién lo consulta más: que

Arriendos pregunte seguido por disponibilidad no justifica mover la disponibilidad a Arriendos, porque eso crearía dos fuentes de verdad.

Se descarta el monolito (Opciones A y B) por tres razones:

- **Ciclos de vida distintos.** El inventario cambia por razones de mantenimiento, altas y bajas; los arriendos cambian por reglas comerciales. En un monolito, cada cambio de uno obliga a desplegar y regresionar el otro.
- **Escalamiento independiente.** Si el servicio de Equipos recibe un aumento de consultas, puede escalarse sin escalar todo el servicio de Arriendos, y un pico de creación de arriendos no obliga a escalar el inventario. Un monolito, incluso modular, tiene un único proceso y un único pool de recursos.
- **Reutilización de Equipos.** El servicio de Equipos puede ser usado por otros servicios además de Arriendos. Mantenerlo independiente permite reutilizarlo sin acoplar directamente su lógica al sistema de arriendos.

Se descarta la base de datos compartida (Opción C) porque rompe la propiedad de los datos: si Arriendos puede escribir la tabla de unidades, la disponibilidad deja de tener un único dueño y puede modificarse sin pasar por las reglas de Equipos. Además, el esquema pasa a ser el contrato entre servicios y no tiene versionado: cualquier cambio de columna puede romper al otro servicio sin aviso, lo que deja un monolito distribuido (dos procesos acoplados como uno). Por último, una base de datos compartida es un punto único de falla y de saturación para ambos servicios, justamente lo que la separación busca evitar.

Se aceptan dos duplicaciones de datos a propósito:

- `equipment_id` **en** `Rental`**.** Arriendos guarda la referencia a la unidad arrendada para mostrar y gestionar el arriendo sin consultar a Equipos en cada lectura. Es una referencia, no una copia del equipo.
- **Copia de los arriendos en Redis** (`rental:{id}` y `rentals:list:*`, con TTL de 60 s). Reduce la carga de la base de datos en las lecturas más frecuentes. La coherencia se mantiene invalidando la clave puntual y todas las claves de listado en cada mutación (`create_rental`, `cancel_rental`, `return_rental`).

Microsoft 2026

Costo aceptado. Se aceptan los siguientes costos:

- **Dependencia de comunicación.** Cuando Arriendos necesita información o una acción de Equipos, debe consultar a ese servicio, lo que agrega latencia y puede provocar errores si Equipos no está disponible.

- **Pérdida de atomicidad.** Si `reservar_unidad` tiene éxito pero el `commit` local falla, la compensación (`liberar_unidad`) puede fallar también y dejar una unidad reservada sin arriendo asociado. Se acepta este riesgo, documentado en el código, a cambio de no implementar una transacción distribuida.
- `equipment_id` **no validable fuera de la reserva.** Sin llave foránea real entre servicios, Arriendos solo comprueba que la unidad existe en el instante de `reservar_unidad`. Si después una unidad se elimina o cambia en Equipos, los arriendos que la referencian quedan con un `equipment_id` colgante, y Arriendos no se entera. En el código actual, `cancel_rental` y `return_rental` llaman a `liberar_unidad` antes de cambiar el estado local, por lo que un arriendo podría quedar `ACTIVE` indefinidamente si Equipos rechaza la liberación. Se propone mitigarlo con tres medidas: Equipos no elimina físicamente unidades ni reutiliza IDs (baja lógica), `liberar_unidad` es idempotente, y un proceso periódico de reconciliación compara las reservas de Equipos con los arriendos `ACTIVE`.
- **Obsolescencia en Redis.** Los datos pueden estar desfasados hasta 60 s, y cada endpoint nuevo que modifique `Rental` debe acordarse de invalidar las claves correspondientes, sin que ningún error avise si se olvida.

Consecuencias. Se introduce comunicación entre servicios, lo que agrega latencia, posibles fallos de red y mayor complejidad operacional en comparación con un monolito. La consistencia entre ambos servicios pasa a ser responsabilidad de la aplicación (compensación, reglas de baja y reconciliación) y no de la base de datos. Esta decisión habría que revisarla si las reservas huérfanas o los arriendos imposibles de cerrar dejan de ser casos raros (convendría una saga formal o eventos), o si el equipo y el tráfico son tan pequeños que operar dos servicios cuesta más de lo que aporta (la Opción B sería entonces la alternativa razonable).

ADR-002 - REST frente a gRPC. REST como cara pública y gRPC como contacto para la conexión.

Estado: Aceptada

Contexto. Una vez elegido la arquitectura de microservicios, se busca implementar un método de comunicación entre servicios y una forma de interactuar con el entorno.

Alternativas consideradas.

- *Opción A* – Implementar websockets como protocolo de comunicación e implementar REST en ambos extremos.
- *Opción B* – Implementar gRPC como protocolo de comunicación y REST como metodo de interacción con el usuario.

Decisión. Se eligió implementar el protocolo gRPC como metodo de comunicación y usar REST como método de interacción con el usuario.

Justificación. El punto que descarta WebSockets es el tipo de interacción: reservar_unidad necesita una confirmación explícita e inmediata (¿se reservó o no?) antes de que create_rental pueda seguir — es RPC, no un flujo de eventos asíncronos. gRPC modela esto de forma nativa: la llamada devuelve una respuesta tipada o lanza un error, con timeouts y códigos de estado estandarizados. WebSockets, en cambio, es un canal de mensajes sin una noción incorporada de "esta respuesta contesta a esa solicitud" ni de tipado — tendríamos que reinventar sobre JSON crudo un protocolo de correlación de mensajes, control de errores y timeouts que gRPC ya trae resuelto, y encima sin el tipado en compilación que si aporta .proto. A eso se suma que gRPC corre sobre HTTP/2 con multiplexado, entregando el payload compacto de Protobuf sin el overhead de mantener una conexión persistente abierta expresamente para esto. WebSockets solo se justificaría si Equipos necesitara notificar a Arriendos de cambios que Arriendos no inició (por ejemplo, una unidad dada de baja por mantenimiento) — un caso que hoy no existe en el flujo descrito. (gRPC Authors 2026b)

Costo aceptado. Se manda todo el patrón interno por request-response síncrono vía gRPC, sacrificando la posibilidad de que Equipos notifique proactivamente a Arriendos de cambios de estado que no fueron iniciados por una llamada de Arriendos. Si ese caso de uso aparece en el futuro, hoy no hay canal para resolverlo sin agregar algo nuevo.

Consecuencias. Mientras la relación entre ambos servicios se mantenga como solicitud-respuesta (reservar/liberar), gRPC sigue siendo la opción correcta y no hay necesidad de introducir infraestructura de conexiones persistentes. Si en el futuro el negocio necesita que Equipos empuje eventos a Arriendos (o a otros consumidores) sin que estos pregunten, esta decisión debería revisarse: ahí sí valdría la pena evaluar WebSockets o, más probablemente, un mecanismo de streaming (gRPC server-streaming o un message broker) en vez de forzar ese patrón sobre el RPC síncrono actual.

ADR-003 - Contrato, versionado y evolución. Evolución de la API y cambios en los contratos.

Estado: Propuesta

Contexto. El sistema ya expone contratos explícitos en ambos bordes: modelos Pydantic/OpenAPI en la API pública (ClientRequest, RentalResponse, etc.) y definiciones .proto en la comunicación interna con Catálogo. Ambos contratos van a cambiar con el tiempo —nuevos campos, nuevos estados, nuevas reglas de negocio— y hay consumidores externos que no se despliegan al mismo tiempo que este servicio. Hace falta una regla explícita de qué cambios son seguros y cómo se comunican los que no lo son, o cada cambio de contrato se convierte en una negociación adhoc.

Alternativas consideradas.

- *Opción A* – Versionado en la URI (/v1/..., /v2/...), rompiendo compatibilidad solo al cambiar de versión.
- *Opción B* – Un solo /v1/ para siempre, evolucionando el contrato solo con cambios aditivos (nunca se rompe nada, nunca se versiona).

Decisión. Versionado explícito en la URI (/v1/...), donde el prefijo de versión solo sube cuando hay un cambio incompatible; los cambios compatibles se agregan dentro de la misma versión. (Protocol Buffers Authors 2026)

Justificación. El código ya sigue esta convención (prefix='/v1/clients', prefix='/v1/rentals'), así que la decisión formaliza lo existente en vez de introducir algo nuevo. Frente a la Opción B, forzar "todo compatible para siempre" no es sostenible: tarde o temprano una regla de negocio exige un cambio real de forma (ejemplo concreto más abajo), y sin una vía de versión nueva ese cambio se aplicaría sobre los mismos consumidores sin aviso, rompiendo integraciones en producción sin que lo noten hasta que fallan.

Costo aceptado. Mantener /v1/ y una eventual /v2/ corriendo en paralelo durante el período de migración implica duplicar parcialmente routers y lógica de traducción entre versiones (o al menos mantener ambos hasta deprecar la vieja), y coordinar con los consumidores un aviso de deprecación con plazo antes de apagar /v1/.

Consecuencias. Se logra establecer diferencias claras entre distintas versiones, y a la vez se permite tener varias versiones de la aplicación activas a la vez, de esta forma no se fuerza a los clientes a cambiarse y se puede tener mayor control sobre quienes usan que versión. Se va a requerir de adapatar los contratos al momento de cambiar de versión y de mantener la documentación y servidores activos para las versiones activas generando un mayor uso de recursos, pero que se puede eliminar una vez se cambien definitivamente de versión. Esta decisión queda como propuesta ya que se tendrá que hacer valida a medida que se vayan desarrollando estas nuevas versiones de la aplicación.

Evolución del contrato gRPC (`equipos.v1`). El paquete `equipos.v1` identifica la versión del contrato Protocol Buffers y es independiente del versionado por URI de la API REST. Por ejemplo, si Equipos necesita informar la categoría de una máquina, se puede agregar `string categoria = 5;` al mensaje `Equipo`, sin cambiar los números ni los tipos de los campos existentes: `id = 1`, `nombre = 2`, `descripcion = 3` y `unidades_disponibles = 4`. Este cambio es compatible con el formato binario: los consumidores antiguos ignoran el campo desconocido y los nuevos consumidores que reciban una respuesta de un servidor antiguo obtienen el valor por defecto de proto3 (`""` para `string`). Por ello, durante una actualización gradual, el consumidor debe tolerar que el campo todavía no tenga un valor informativo Protocol Buffers Authors 2026.

Los números de campo son parte de la identidad binaria del contrato: no se deben renumerar ni reutilizar, ni cambiar el tipo de un campo existente por otro incompatible. Si un campo deja de utilizarse, se elimina de la definición y su número y nombre se declaran como `reserved`; por ejemplo, `reserved 5;` y `reserved "categoria";` si posteriormente se retira `categoria`. Esto evita que una modificación futura asigne accidentalmente esa etiqueta a otro dato y haga que mensajes antiguos se interpreten de forma incorrecta Protocol Buffers Authors 2026. Cambios incompatibles, como alterar la semántica de un campo, cambiar su tipo o retirar un RPC utilizado, requieren un contrato nuevo, por ejemplo el paquete `equipos.v2`, y una migración coordinada que mantenga disponibles ambas versiones mientras los consumidores actualizan sus stubs.

Los cambios se comunican versionando juntos el archivo `.proto` y el código generado en el repositorio, describiendo la modificación y su compatibilidad en las notas de la versión y ejecutando pruebas de contrato para los consumidores conocidos, incluido el cliente Node.js. Antes de retirar una versión incompatible, se avisa a los consumidores y se establece un período de transición. El paquete `equipos.v1` ya está definido en el contrato actual; la adopción de `equipos.v2` queda reservada para un cambio incompatible real.

ADR-004 - Resiliencia ante fallas. Uso de timesout y respuestas HTTP.

Estado: Aceptado

Contexto. Algunos de las llamadas de la API de Arriendo requieren de utilidades de la API de Equipo, pero para que estas llamadas se resuelvan de manera exitosa se requiere de una conexión estable entre microservicios. Para esto se requeiere evaluar los metodos de control y de timesout que se ocuparan para manejar estos casos de error.

Alternativas consideradas.

- *Opción A* – Sin timeout explícito ofrece una implementación mínima, cero código adicional pero basta con que una sola instancia lenta de Equipos para dejar conexiones de Arriendos esperando.
- *Opción B* – Timeout explícito y corto en la llamada gRPC, con fail-fast hacia el cliente ofrece que no se queden llamadas en espera indefinida, sino que se cortan y entregan una respuesta acorde a las métricas expuestas en los contratos.
- *Opción C* – Servir datos de caché como fallback cuando Equipo no responde puede ayudar a agilizar las consultas de llamado, pero no permite que se usen consultas del tipo POST, ya que el cache no asegura que los datos se mantengan íntegros o se respete lo que hay en la base de datos.

Decisión. Timeout explícito y corto en la llamada gRPC, con fail-fast hacia el cliente, reaccionando con un código de error preestablecido por el contrato OPENAPI, manteniendo la integridad segura y evitando tiempos de espera eternos. (gRPC Authors 2026a)

Justificación. Dado que los endpoints de Arriendo requieren de escribir en la base de datos de Equipos, no es posible usar la cache como forma de resiliencia, ya que esta no asegura la escritura de los datos alterados en la base de datos de Equipo, por lo que al momento de actualizarse podrían haber fallas e incoherencias si es que hubieran dos o mas servicios conectados, de esta forma se descarta la opción C. Y la opción A queda claramente descartada, ya que si no hay un tiempo máximo para las peticiones, estas se acumularían en una cola que no sabe cuando va a volver a activarse el microservicio de Equipos, generando posibles incosistencias entre bases de datos y provocando demora y fallas a lo largo de la aplicación.

A su vez, tener códigos diferenciados para cada falla posible le da al consumidor información adicional del error y permite actuar en consecuencia. Tener estas respuestas declaradas entrega el fallo y las maneras de proceder.

Costo aceptado. Con fail-fast, se rechazan operaciones que quizás habrían tenido éxito si Arriendos hubiera esperado un par de segundos más — se sacrifica disponibilidad "optimista" por previsibilidad y por proteger la capacidad del propio servicio. Además, un timeout mal calibrado (demasiado corto) puede generar falsos 504 bajo carga normal de Equipos si su latencia p99 se acerca al valor elegido; el valor del timeout debe fijarse con datos reales de latencia de Equipos, no arbitrariamente.

Consecuencias. Los endpoints create_rental, cancel_rental y return_rental deben envolver la llamada gRPC con un timeout explícito y traducir explícitamente las excepciones de gRPC (DEADLINE_EXCEEDED → 504, UNAVAILABLE → 503) en vez de dejar que una excepción no capturada llegue como 500 genérico. Si en el futuro el volumen de fallos de Catálogo bajo carga alta resulta frecuente, habría que revisar esta decisión e incorporar un circuit breaker (abrir el circuito tras N fallos consecutivos y devolver 503 inmediatamente sin ni siquiera intentar la llamada) para evitar seguir golpeando a un Equipos ya degradado. (gRPC Authors 2026a; Fielding, Nottingham y Reschke 2022)

4 Experimento

4.1 Objetivo e hipótesis

El objetivo del experimento es evaluar el efecto de Redis sobre la latencia de las consultas de arriendos. Para ello, se compara el sistema sin caché con tres patrones de acceso cuando Redis está habilitado: consultas con acierto de caché (*HIT*), consultas con fallo de caché (*MISS*) y una combinación de ambas.

Se plantea que los HIT reducirán la latencia, porque la respuesta podrá obtenerse desde Redis sin consultar la base de datos. En cambio, se espera que los MISS aumenten la latencia respecto de una consulta sin caché: la aplicación deberá consultar Redis, acceder luego a la base de datos y almacenar el resultado para futuras solicitudes.

Como motivación para evaluar una caché, se considera el supuesto de que las consultas reales podrían concentrarse en un conjunto reducido de recursos, de manera similar a una distribución de Pareto. El experimento no verifica esa distribución en usuarios reales; compara patrones de acceso definidos para observar cómo cambia el rendimiento según la frecuencia de HIT y MISS.

4.2 Diseño y métricas

Se definieron los cuatro escenarios de la Tabla 1.

La configuración predeterminada del programa es de 10 000 solicitudes por ejecución, 15 trabajadores concurrentes, 100 solicitudes de calentamiento y 10 repeticiones por escenario.

Cuadro 1: Escenarios del experimento de caché.

Escenario	Configuración	Patrón de consultas
A	Sin Redis	Consultas a la base de datos.
B	Redis habilitado	Consultas repetidas a un recurso previamente almacenado en caché.
C	Redis habilitado	Consultas a recursos distintos, destinadas a producir MISS.
D	Redis habilitado	Consultas repetidas a varios recursos, destinadas a combinar HIT y MISS.

El análisis estadístico disponible contiene diez repeticiones para cada uno. No se conservaron en el archivo de análisis los parámetros efectivos de ejecución, por lo que las cifras anteriores describen los valores predeterminados del código.

Se midieron la latencia media, la mediana de latencia (P50), los percentiles P95 y P99, y el rendimiento en solicitudes por segundo (RPS). Para resumir cada escenario se utiliza la mediana de sus diez ejecuciones. Una latencia menor representa una mejora; para RPS, la mejora corresponde a un valor mayor.

4.3 Resultados

La Tabla 2 presenta la mediana entre repeticiones para cada métrica. Cada percentil fue calculado primero dentro de una ejecución y luego resumido mediante la mediana de las diez ejecuciones.

Cuadro 2: Resultados por escenario: mediana de diez ejecuciones.

Métrica	A: sin caché	B: HIT	C: MISS	D: mixto
Latencia media (ms)	59,88	55,44	64,74	59,90
P50 (ms)	33,41	24,25	59,12	51,89
P95 (ms)	197,51	191,98	114,14	150,92
P99 (ms)	359,35	321,06	199,73	262,71
Rendimiento (RPS)	250,17	270,25	231,50	250,16

Frente al escenario sin caché, el escenario HIT redujo el P50 en 27,4 % y el P99 en 10,7 %; su rendimiento aumentó en 8,0 %. En el escenario MISS, el P50 aumentó en 77,0 % y el rendimiento disminuyó en 7,5 %. El escenario mixto prácticamente mantuvo la latencia media y el rendimiento del escenario A, aunque presentó un P50 mayor y percentiles P95 y P99 menores.

4.4 Prueba estadística

Se aplicó la prueba de rangos con signo de Wilcoxon a las diez repeticiones, con un nivel de significancia de $\alpha = 0{,}05$. Para las latencias se evaluó la hipótesis unilateral de que cada escenario con Redis tuviera valores menores que el escenario sin caché. Para RPS, donde un valor mayor es favorable, se evaluó la hipótesis de mayor rendimiento.

En la Tabla 3, las celdas verdes representan valores $p < 0{,}05$, por lo que se rechaza la hipótesis nula para esa métrica. Las celdas rojas representan valores $p \geq 0{,}05$, por lo que no existe evidencia suficiente para rechazar la hipótesis nula.

Cuadro 3: Valores p de Wilcoxon frente al escenario A.

Comparación	Media	P50	P95	P99	Mayor RPS
B: HIT	0,0527	**0,0010**	0,2783	**0,0322**	**0,0322**
C: MISS	0,9990	1,0000	**0,0010**	**0,0010**	0,9990
D: mixto	0,3848	1,0000	**0,0010**	**0,0010**	0,3848

En el escenario HIT, se rechaza la hipótesis nula para P50, P99 y RPS. Esto indica evidencia estadística de que los HIT reducen la latencia habitual, reducen parte de las latencias extremas y aumentan el rendimiento. La mejora de la latencia media quedó cerca del umbral, pero no fue significativa ($p = 0{,}0527$).

Los resultados de los escenarios MISS y mixto deben interpretarse con cautela, porque los percentiles P95 y P99 disminuyeron, mientras que el P50 aumentó o no mostró una mejora. Además, se realizaron varias comparaciones sin ajustar el nivel de significancia por comparaciones múltiples.

4.5 Interpretación y limitaciones

Los resultados muestran que Redis mejora el rendimiento cuando las consultas producen HIT. En este escenario, se observaron mejoras estadísticamente significativas en el P50, el P99 y el rendimiento medido en RPS. La latencia media también disminuyó, aunque su valor $p = 0{,}0527$ quedó levemente por sobre el nivel de significancia definido, por lo que no se rechaza la hipótesis nula para esa métrica.

En el escenario MISS, la latencia habitual empeoró, como se esperaba. El P50 aumentó porque la aplicación primero consulta Redis y, al no encontrar el dato, debe realizar además una consulta a PostgreSQL. Esto agrega una operación adicional respecto del escenario sin caché.

Sin embargo, los percentiles P95 y P99 del escenario MISS fueron menores y presentaron significancia estadística. Este resultado no permite concluir por sí solo que los MISS mejoren las latencias extremas. Los escenarios no aislaron completamente todas las variables: los patrones de consulta, la cantidad de datos, la planificación del sistema operativo, la actividad

de entrada y salida, y la carga de PostgreSQL pueden afectar especialmente las solicitudes más lentas.

El escenario D, que combina HIT y MISS en una proporción configurada de 50/50, presentó resultados cercanos al escenario sin caché en la latencia media y en RPS. Su P50 fue mayor que el escenario A, lo que sugiere que el costo de los MISS puede compensar el beneficio de los HIT cuando ambos ocurren con una frecuencia similar. Como Redis mantiene los datos en memoria, el acceso a un HIT es más rápido que una consulta a la base de datos; no obstante, un MISS requiere ambas operaciones y puede afectar el resultado total.

Una limitación del experimento es que solo se evaluó una proporción mixta de 50/50. Sería útil repetir las mediciones con distintas tasas de HIT, por ejemplo 25/75, 50/50, 75/25 y 90/10, para identificar desde qué proporción de HIT el uso de Redis comienza a ser ventajoso para este sistema.

4.6 Conclusiones

Los resultados sugieren que el uso de Redis es conveniente cuando se espera que la proporción de operaciones HIT sea considerablemente mayor que la de operaciones MISS. En particular, el escenario HIT presentó mejoras estadísticamente significativas en el P50, el P99 y el rendimiento medido en RPS. Por el contrario, el escenario MISS aumentó la latencia habitual, debido a que la aplicación debe consultar Redis y posteriormente acceder a la base de datos.

Esta conclusión es compatible con la idea de que las consultas reales pueden seguir una distribución de tipo Pareto, donde una cantidad reducida de recursos concentra una proporción importante de las solicitudes. Sin embargo, la distribución de Pareto no fue verificada empíricamente en este experimento, por lo que debe considerarse únicamente como una motivación para el uso de la caché.

Los resultados proporcionan indicios de que Redis puede mejorar el rendimiento cuando la mayoría de las operaciones corresponde a HIT. No obstante, el experimento no permite establecer un umbral preciso de proporción HIT a partir del cual la caché comienza a ser beneficiosa. Para determinarlo sería necesario repetir las mediciones utilizando distintas proporciones de HIT y MISS, manteniendo constantes los datos, la carga y las condiciones de ejecución.

5 Consideraciones de contexto

Se eligió la seguridad y privacidad de los datos como factor de contexto para el diseño del sistema. La API REST de Arriendos administra información de clientes, como nombre, correo electrónico e historial de arriendos, por lo que se implementó autenticación mediante *API Key* y autorización según permisos.

La clave `apikey_cliente` permite únicamente operaciones de lectura, mientras que `apikey_admin` permite también operaciones de escritura, como crear clientes, registrar arriendos, cancelarlos o retornarlos. De esta forma, un consumidor que solo necesita consultar información no puede modificar los datos de clientes, arriendos ni la disponibilidad de equipos mediante la API REST.

Además, cada microservicio mantiene su propia base de datos y expone solo las operaciones necesarias para cumplir sus responsabilidades. El servicio de Arriendos no accede directamente a la base de datos de Equipos, y el servicio de Equipos no recibe datos de clientes: únicamente procesa identificadores de equipos y operaciones relacionadas con su disponibilidad mediante gRPC.

La comunicación gRPC está destinada a la integración interna entre Arriendos y Equipos, mientras que la API REST constituye el punto de entrada para los consumidores externos. El servicio de Equipos actúa como fuente de verdad para el catálogo y la disponibilidad, por lo que Arriendos debe solicitar la reserva o liberación de una unidad mediante gRPC antes de actualizar el contrato local.

Se eligió la autenticación mediante API Key por su sencillez para identificar consumidores conocidos y asignarles permisos de lectura o administración. Frente a HTTP Basic, evita incorporar la gestión de usuarios y contraseñas para una integración que solo requiere distinguir estos dos roles. Una solución basada en tokens Bearer permitiría incorporar mecanismos de expiración y permisos más detallados, pero requeriría gestionar su emisión y validación, aumentando la complejidad del prototipo. Por estas razones, API Key se considera suficiente para el alcance del proyecto.

Las claves se obtienen mediante las variables de entorno ADMIN_API_KEY y CLIENT_API_KEY, separando las credenciales del código fuente y permitiendo configurarlas según el entorno de ejecución. La API responde con HTTP 401 cuando la clave está ausente o es inválida, y con HTTP 403 cuando una clave válida de solo lectura intenta realizar una operación de escritura.

6 Instrucciones de ejecución

Las siguientes instrucciones asumen que Docker y Docker Compose se encuentran instalados y que los comandos se ejecutan desde la raíz del proyecto.

6.1 Ejecución de los servicios

Para construir las imágenes y levantar los servicios definidos en el archivo `docker-compose.yaml`, ejecutar:

```
docker compose -f docker-compose.yaml up --build
```

Para detener los contenedores asociados al proyecto, ejecutar:

```
docker compose down
```

Si además se desea eliminar los volúmenes asociados a los servicios, lo que implica eliminar los datos persistidos en ellos, ejecutar:

```
docker compose down -v
```

6.2 Ejecución de las pruebas

Las pruebas pueden ejecutarse mediante la configuración definida en `docker-compose.test.yaml`. Para ello, ejecutar:

```
docker compose -f docker-compose.test.yaml up --build
```

Una vez finalizadas las pruebas, las bases de datos pueden permanecer activas. Para detener los servicios y eliminar los volúmenes generados durante la ejecución de las pruebas, ejecutar:

```
docker compose down -v
```

6.3 Verificación de contenedores activos

Para comprobar si existen contenedores en ejecución, utilizar:

```
docker ps
```

Si se identifica algún contenedor que permanezca activo y deba ser detenido, puede utilizarse el siguiente comando, reemplazando `<id_contenedor>` por el identificador correspondiente:

```
docker stop <id_contenedor>
```

Una vez detenido el contenedor, si se desea eliminarlo junto con los volúmenes anónimos asociados, utilizar:

```
docker rm -v <id_contenedor>
```

Esta última operación elimina el contenedor y los volúmenes anónimos asociados a él.

7 Reflexión final

Este trabajo fue especialmente provechoso para el grupo porque permitió aplicar tecnologías y prácticas que hasta ahora conocíamos principalmente de forma teórica, o que no habíamos utilizado directamente en un proyecto integrado. En particular, trabajar con OpenAPI y contratos gRPC nos permitió comprender la importancia de definir explícitamente cómo se comunican los servicios antes de implementar su lógica. Los contratos facilitaron que cada parte del sistema tuviera una interfaz clara y redujeron la incertidumbre al momento de integrar los componentes.

Para algunos integrantes, esta fue también la primera experiencia utilizando Redis como caché. El experimento permitió ir más allá de asumir que una caché siempre mejora el rendimiento: observamos que los HIT pueden reducir la latencia y mejorar el rendimiento, mientras que los MISS agregan un costo adicional. Aunque hemos realizado experimentos en otros cursos, normalmente asociados a estructuras de datos o algoritmos, aplicar una metodología de medición a una decisión arquitectónica fue una experiencia particularmente útil. Nos permitió fundamentar una elección tecnológica con datos, reconocer las limitaciones de los resultados y entender que el rendimiento depende del patrón real de uso.

El diseño basado en microservicios también benefició la organización del equipo. La separación entre Arriendos y Equipos permitió dividir las tareas de manera más clara, avanzar en paralelo y reducir el riesgo de interferir con el trabajo de otros integrantes. La mayor coordinación se concentró en los puntos de integración, especialmente en la definición de los contratos, las llamadas gRPC y el comportamiento ante errores. Esto agilizó el desarrollo, aunque también mostró que dividir un sistema en servicios introduce responsabilidades adicionales, como mantener contratos compatibles, manejar fallas de red y verificar la consistencia entre servicios.

En síntesis, el principal aprendizaje del grupo fue que las tecnologías de integración no solo resuelven problemas técnicos: también influyen en la forma de organizar el trabajo, coordinar decisiones y justificar una solución. Este proyecto nos permitió relacionar diseño, implementación, experimentación y trabajo colaborativo en una misma experiencia.

Referencias

Fielding, Roy, Mark Nottingham y Julian Reschke (jun. de 2022). *HTTP Semantics*. RFC 9110. Internet Engineering Task Force. URL: https://www.rfc-editor.org/rfc/rfc9110.html.

gRPC Authors (2026a). *Deadlines*. URL: https://grpc.io/docs/guides/deadlines/ (visitado 25-09-2026).

— (2026b). *gRPC Documentation*. URL: https://grpc.io/docs/ (visitado 25-09-2026).

Microsoft (2026). *Data Considerations for Microservices*. Azure Architecture Center. URL: https://learn.microsoft.com/en-us/azure/architecture/microservices/design/data-considerations (visitado 25-09-2026).

Protocol Buffers Authors (2026). *Language Guide (proto 3): Updating a Message Type*. URL: https://protobuf.dev/programming-guides/proto3/#updating (visitado 25-09-2026).